

1. Temel Bit İşlem Operatörleri

AND Operatörü (&)

- İki sayının **her bir bitini** karşılaştırır. İkisi de 1 ise sonuç 1, aksi halde 0 olur.

```
int a = 5; // 0101 (binary)
int b = 3; // 0011 (binary)
int result = a & b; // 0001 (binary) = 1
```

OR Operatörü (|)

- İki sayının **her bir bitini** karşılaştırır. Biri bile 1 ise sonuç 1, aksi halde 0 olur.

```
int a = 5; // 0101 (binary)
int b = 3; // 0011 (binary)
int result = a | b; // 0111 (binary) = 7
```

XOR Operatörü (^)

- İki sayının **her bir bitini** karşılaştırır. Biri 1 ve diğeri 0 ise sonuç 1, aksi halde 0 olur.

```
int a = 5; // 0101 (binary)
int b = 3; // 0011 (binary)
int result = a ^ b; // 0110 (binary) = 6
```

NOT Operatörü (~)

- Tek bir sayının **tüm bitlerini ters çevirir** (0ları 1 yapar, 1leri 0 yapar).

```
int a = 5; // 0101 (binary)
int result = ~a; // 1010 (binary, 2'nin tümleyeni) = -6
```

1'in tümleyeni, bir binary (ikili) sayının tüm bitlerini tersine çevirmek anlamına gelir.

2'nin tümleyeni, bilgisayarlarda negatif sayıları temsil etmek için kullanılan bir yöntemdir. Bu yöntem, bir binary sayının negatifini bulmak için kullanılır.

Örnek: -6'yı 4 bit ile temsil etmek

Pozitif 6'yı binary olarak yaz:

6 = 0110

Tüm bitleri ters çevir (1'in tümleyeni):

0110 → 1001

1 ekle:

1001 + 1 = 1010

Binary Toplama Kuralları

1. $0 + 0 = 0$
2. $0 + 1 = 1$
3. $1 + 0 = 1$
4. $1 + 1 = 10 \rightarrow$ Sonuç 0, elde 1.

Binary Çıkarma Kuralları

1. $0 - 0 = 0$
2. $1 - 0 = 1$
3. $1 - 1 = 0$
4. $0 - 1 = 1$ (borç alınır)

Bit Kaydırma Operatörleri

Sol Kaydırma (<<)

- Bitleri sola doğru kaydırır. Sağdan sıfır eklenir.
- Her bir kaydırma işlemi sayıyı 2 ile çarpar.

```
int a = 5; // 0101 (binary)
int result = a << 1; // 1010 (binary) = 10
```

Sağ Kaydırma (>>)

- Bitleri sağa doğru kaydırır. İşaretli sağ kaydırmada (signed), soldan işaret biti (0 veya 1) eklenir.
- Her bir kaydırma işlemi sayıyı 2'ye böler.

```
int a = 5; // 0101 (binary)
int result = a >> 1; // 0010 (binary) = 2
```

Örnek:

→4'ü 2 bit sola kaydır: $4 \ll 2 = 16$

$00000100 \ll 2 = 00010000$

→20'yi 2 bit sağa kaydır:

$20 \gg 2 = 5$

$00010100 \gg 2 = 00000101$

→uint8_t value = 01000000; //4 kere sola kaydır

- Eğer uint8_t value = 01000000 (decimal 64) olarak başlarsanız ve 4 kere sola kaydırırsanız, sonuç **0** olur.
- Çünkü **8 bitlik bir sayı** olduğunda, daha fazla sola kaydırma, **en sağdaki bitlerin kaybolmasına** yol açar ve **0'a** dönüşür.

Belirli bir biti 1 yapma

Örneğin A değişkeninin 2. bitini SET etmek için

$A = A \mid 4;$

$A = A \mid (1 \ll 2);$

$A \mid= (1 \ll 2);$

$A \mid= (1 \ll 2); \rightarrow A = A \mid (1 \ll 2)$

A

0	1	0	1	0	0	0	1
---	---	---	---	---	---	---	---

$1 \ll 2$

0	0	0	0	0	1	0	0
---	---	---	---	---	---	---	---

$A \mid= (1 \ll 2)$

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Örneğin A değişkeninin 4. bitini sıfırlamak için

```
A &= ~ (1<<4);
```

```
A = A & ~(00010000)
```

```
A = A & 11101111
```

A

0	0	0	1	0	0	0	1
---	---	---	---	---	---	---	---

$\sim(1<<4)$

1	1	1	0	1	1	1	1
---	---	---	---	---	---	---	---

$A \&\sim(1<<4)$

0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---

XOR ile biti tersine çevirme

```
REG_A ^= (1<<2);
```

REG_A

0	0	0	0	1	1	1	1
---	---	---	---	---	---	---	---

$(1<<2)$

0	0	0	0	0	1	0	0
---	---	---	---	---	---	---	---

$REG_A \wedge (1<<2)$

0	0	0	0	1	0	1	1
---	---	---	---	---	---	---	---

Bit Bazlı İşlemler

Programlarda bir register'ın belirli bir bitinin 0 veya 1 olduğunu kontrol etmek gerekebilir

```
while ( REG_A & (1<<7) == (1<<7))  
{  
    // REG_A 7. biti 1 olduğu sürece çalışır  
}
```